AARHUS UNIVERSITY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

COMPUTER ENGINEERING

Term:	FALL 2022 (re-exam)
Examination:	Introduction to Programming for Computer Engineers
Date:	24 May 2023
Duration:	3 hours, 9:00 – 12:00
Room:	room number(s)

1 cover plus paper for draft notes will be handed out.

REMEMBER to state name and study number on all pages of your submission.

Remember to show us an ID when returning the present exam form. We will then mark your name on our master list.

- This document is split into 5 sections focusing on different topics of the lecture. Each section contains up to 5 questions. You have 180 minutes to answer all of them. In addition, there is one challenging bonus question that you can work on if you get the time, since it will be possible to get the highest mark even without answering the bonus question, but it can compensate for mistakes you have done in the first 5 sections. Strategically, you should first scan it quickly in its entirety, and identify the groups of questions that seem easiest to you. ***You should answer those first*** and then move on to the others.
- For each question it is indicated how and where the answer must be provided. There are multiple choice questions, questions that require you to write C code and drawings. Only what is written or drawn inside the specified areas is considered as your answer.
- **If you are in doubt what is expected, please ask!**
- The multiple choice questions may have any combination of correct answers. All choices may be correct, none or anything in between.
- Each question has a weight **[w]**, where **w** is an integer number, specifying the number of points awarded for that question. You can find the weight at the top right corner of each question. The total number of points of all questions together is 100.
- At the end of the exam paper (Section 6) you find one bonus question worth 10 points. You are ***not*** required to solve it and can attain the full score of 100 points ***without*** it. It is harder to solve, but it can be used to compensate for mistakes in answers to other questions.

1 Types and Syntax

Question 1.1 **[3]**

What type do the following values belong to? Indicate your answer by ticking the boxes on the right.

	`float`	`float[2]`	`char`	`char[10]`	other or not a correct value
• `6.02e-2`	☒	☐	☐	☐	☐
• `"4"`	☐	☐	☐	☒	☐
• `'7'`	☐	☐	☒	☐	☐
• `{5.0, 3.2 }`	☐	☒	☐	☐	☐

Question 1.2 **[3]**

What is the difference between debugging and linking? (tick the correct box):

- ☐ Debugging generates an object file, linking an executable file
- ☐ Debugging creates an executable file, linking an archive file
- ☐ Nothing, they are the same
- ☒ None of the above

Question 1.3 **[3]**

A function `volume` calculates and returns the volume of a square as a double. The input is given as two doubles: `w` and `h`, where `w` is the width of the square, `h` is the heigth. Complete the function signature below.

```
1
2 double volume ( double w , double h ) {
3
4   return w*h;
5 }
```

Question 1.4 **[3]**

The function `half` takes a pointer to an float and halfs the float value which it points to. The function does not return any value. Complete the function signature and function body below, so that the main function prints `3.5` when executed.

```
void
~~float~~ half ( float * f ) {
  ~~return~~ *f = *f/2 ;
}

int main () {
  float r = 7.0;
  half(&r);
  printf("%f", r);
  return 0;
}
```

Question 1.5 **[3]**

Fill in the type definition below for a struct defining a type called `kitchen`. It must contain the fields `cups` of type `int`, `plates` of type `int`, and `nextKitchen` of type `kitchen` pointer.

```
typedef struct kitchen {
    int cups;
    int plates;
    kitchen *nextKitchen;
  } kitchen;
```

2 Basic Programs

Question 2.1 [4]

Let x, y, z be integer variables declared by

```
int x;
int y;
int z;
```

Which assignment (a), (b), (c), (d) or (e)inserted for the question mark in the program fragment below makes the following program fragment print out 6:

```
  x = 2;
  y = 6;
  ?
  z = 2 * x + y;
  printf("%d", z);
```

Answer:

(a) `y = 3*x;` ☒
(b) `x = y;` ☐
(c) `y = 6;` ☒
(d) `z = 6;` ☐
(e) `none of the above` ☐

Tick the box on the right of the assignment to indicate by which assignment to replace the question mark in the program fragment. More than one expression may be correct.

Question 2.2 [4]

Consider the following program fragment

```
int i;
int b[] = { 0,1,1,2,3,5,8,13 };
for (i = 1; i <= 3; i = i + 2)
   b[i] = b[i] + b[i + 1];
```

Fill in the table below with the values stored in array b after the program has been executed.

Answer: b =

0	2	1	5	3	5	8	13

Question 2.3 **[4]**

Consider this program fragment

```
int c = a * b;
int d = 3;
if (b == 1)
   c = 1;
if (a == 0 || b == 1)
   d = 2;
d = c + d;
printf("%d", d);
```

For the values of variables `a` and `b` as indicated below what is the resulting value of variable `d` (i.e. what is printed out)?

a	b	d				
1	1	-1 ☐	0 ☐	1 ☐	2 ☐	someting else ☒
1	0	-1 ☐	0 ☐	1 ☐	2 ☐	someting else ☒
0	1	-1 ☐	0 ☐	1 ☐	2 ☐	someting else ☒
0	0	-1 ☐	0 ☐	1 ☐	2 ☒	someting else ☐

Tick the box next to the correct value for `d` with respect to the initial values of `a` and `b` as stated in the first two columns of the table.

Question 2.4 **[6]**

When running the following program:

```
1 int j;
2 int arr[] = { 0,1,1,2,3,5,8,13 };
3 for (i = 0; i <= 8; i = i + 1)
4    arr[i] = arr[i] * 2;
```

it has error(s)! What is/are the error(s) and how will you change to program so that it can run without errors (include the line number(s) where you will make the change(s))?

The error is: in the loop, the last value of i is 8, generating an "out of index" error. AND i is not defined

It is corrected by: line 3: for (int i=0; i<8; i=i+1)

Question 2.5 **[6]**

Consider the program fragment below.

```
1  int x;
2  int y;
3  int z;
4
5  x = ?;    y = ?;    z = ?;
6
7  switch (y) {
8     case 0:
9        x = y + x;
10       z = z + x;
11       break;
12    case 1:
13       z = z + y;
14       break;
15    default:
16    {
17       int y;
18       y = x - z;
19       x = y - z;
20    }
21    break;
22 }
23
24 printf("[%d, %d, %d]", x, y, z);
```

For each of the following assignments for variables `x`, `y`, `z` in line 5, write the three integer numbers that will be printed when the program is executed.

(1) `x=0; y=2; z=4;`

Answer: [-8 , 2 , 4]

(2) `x=1; y=x; z=2*y;`

Answer: [1 , 1 , 3]

3 While Programs

Question 3.1 **[6]**

You are given the declarations of variables `j` and `x`, and the initialisation of them.

```
int j;
int k;
j = -3;
k = 7;
```

You are given the following while-loop:

```
while (j <= 0) {
   k = k + j;
   j = k + 1;
}
printf("%d", k);
```

Fill in the blanks below so that the resulting for-loop computes the same values for `j` and `x` as the while-loop above. You are only allowed to use variable declarations, assignments and conditions that already occur in the while-loop.

```
int j;

int k;

k = 7;

for (j = -3; j <= 0; j = k+1) {

   k = k + j;

}
```

What does the program print out?

4

Question 3.2 **[4]**

What do the following program fragments print out at the end of their execution?

Program fragment 1:

```
int x = 2;
for (int i = 0; i < x; i = i + 1)
   x = i + x;
printf("%d \n", x);
```

Program fragment 2:

```
int x = 3;
for (int i = 1; i <= 6; i = i + 1)
   x = i;
printf("%d", x);
```

Indicate your answer by ticking the column with the correct result in the table below. The column 'no result/other' signifies that no value is printed at all (indefinite loop) or another value.

	1	2	3	4	5	6	7	8	9	no result/other
Program fragment 1										X
Program fragment 2						X				

Question 3.3 **[6]**

Consider the following program fragment.

```
int x;
int xi;
int xi_1;
xi_1 = 1;
xi = 3;
scanf("%d", &x);

if (x <= 0)
   abort();

//_______________________  /* Precondition goes here */

while (xi < 2 * x) {

   //  _______________________  /* Loop Invariant goes here */

   xi_1 = xi;
   xi = xi_1 + 1;
}

// _______________________  /* Postcondition goes here */
```

Tick the following boxes such that the property specified for the column holds (as an assertion) at the program location specified by the row. Tick all true statements for each row (from 0 to 6 ticks).

	`xi_1 < xi`	`xi < 2*x`	`x <= 0`	`x > 0`	`xi = xi_1+1`	`xi > 2*x`
Precondition	☒	☐	☐	☒	☐	☐
Loop Invariant	☒	☒	☐	☒	☐	☐
Postcondition	☒	☐	☐	☒	☒	☐

Question 3.4 **[8]**

For each program below, let N be any positive integer. After executing each program, what is the value of v as a function of N?

Program 1:

```
int v = N;
for (int i = N; i >= 1; i = i - 1) {
   v = v + 1;
}
```

Program 2:

```
int v = 0;
for(int j = 1; j <= 2; j = j + 1) {
  for(int i = 1; i <= N; i = i + 1) {
    v = v + 1;
  }
}
```

Program 3:

```
int v = 1;
int x = 2;
for (int i = 1; i <= N; i = i + 1) {
   v = v * x;
}
```

Program 4:

```
int i;
int z;
int v = 0;
z = 1;
for (i = 1; i <= N; i = i + 1) {
   v = v + 1;
}
for (i = 1; i <= N; i = i + 1) {
   v = v + 1;
   z = v * 2;
  }
```

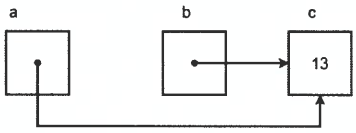
Tick the boxes below to indicate the value of v as a function of N (column) for each program (row). *Hint*: exactly one answer is correct per program.

	N	2*N	N^2	$\sum i_{i=1}^{N}$	2^N	2^{N^2}
Program 1	☐	☒	☐	☐	☐	☐
Program 2	☐	☒	☐	☐	☐	☐
Program 3	☐	☐	☐	☐	☒	☐
Program 4	☐	☒	☐	☐	☐	☐

4 Pointer Programs

Question 4.1 **[3]**

The following pictures (in the notation from the lecture) shows three variables (`a`, `b` and `c`) and their values.



Fill in the rest of the two lines so that the declaration and assignment results in the illustration:

```
int c____ = 13_____ ;
int* b____ = &c_____ ;
int* a____ = &c_____;
```

Hint: Some part of the lines might be empty

Question 4.2 **[4]**

Consider the following program fragment:

```
int x;
int y;
int z;
int* w;
int* q;
x = 2;
y = 2*x;
z = 1;
w = &x;
q = &x;
w = &y;
*q = z;
*w = *w + x +*q;
*q = x + y + z + *w;
printf("x=%d, y=%d, z=%d", x, y, z);
```

What does the program print when it is executed?

Answer: X= 14, y= 6, Z=1

Question 4.3 **[6]**

Consider the following program fragment:

```
1  typedef struct node {
2    int value;
3    struct node* next;
4  } node;
5
6  node* new_node(int v, node* n) {
7     node* new_n = (node*)malloc(sizeof(node));
8     new_n->value = v;
9     new_n->next = n;
10    return new_n;
11 }
12
13 node* func4_3(node* n) {
14    if (n == NULL) {
15       return NULL;
16    }
17    else {
18       n->value = n->value * 2;
19       return n;
20    }
21 }
22
23 void output(node* n) {
24    if (n != NULL) {
25       printf("%d, ", n->value);
26       output(n->next);
27    }
28 }
29
30 int main () {
31    node* a = new_node(4,
32              4,3          new_node(6, NULL));
33    a = func(a);
34    output(a);
35    a = func(a);
36    output(a);
37 }
```

What is the output of the program?

line 34: 8, 6,

line 36: 16, 6,

5 Recursive Programs

Question 5.1 **[4]**

You are given the following program fragment

```
int recur(int n) {
   assert(n >= 0 && n < 1000);
   if (n == 0 || n == 1)
      return 13;
   else
      return recur(n - 1) + recur(n - 2);
}
```

Will the function always work (i.e. terminate and give an answer)?If so, why? If not, what is the problem?

yes. The basecase is 0 and 1 and the recursive part is done when n>1 and subtracts 1 and 2, i.e. always ending in the base case.

Question 5.2 **[4]**

What is the missing assertion and base case of the following function for computing the sum of the numbers from 4 to `n` that are fibonacci numbers by calling, for instance, `k = sumFib(47);` (`fib(f)` is a function that returns if `f` is a fibbonacci number? (the first fibonacci numbers are 1, 1, 2, 3, 5, 8, 13, in general it is the sum of the two previous fibonacci numbers; the first and second are 1 and 1)

```
int sumFib(int n) {
  ____________________________;
  ____________________________;
  else {
    if (fib(n))
      return n + sumFib(n-1);
    else
      return sumFib(n-1);
    }
}
```

Tick the appropriate program fragment to complete the if-statement in function `sum`.

- ☐ `assert(n == 0);`
  `if(n == 0) return 0;`
- ☐ `assert(n >= 4);`
  `if(n == 0) return 0;`
- ☐ `assert(fib(n));`
  `if(fib(n)) return n;`
- ☒ `assert(n >= 4);`
  `if(n == 4) return 0;`

Question 5.3 **[6]**

You are given the following declaration of a list datatype, where the empty list corresponds to the pointer value `NULL`, the field `data` stores the value at a node of the tree, and `next` is the rest of the list.

```
typedef struct node
{
  int data;
  struct node* next;
} node;
```

Complete the missing code in the recursive `countFib` function so that it returns the number of all nodes in the list `l` where the `data` field is a fibonacci number. You can assume that there is a function called `fib(f)` that returns the value `f` if `f` is fibbonacci number, zero (`0`) if not (e.f. `fib(8)` returns `8`; `fib(7)` returns `0`).

```
int countFib(node *l) {


  if( l == NULL )


    return 0 ;


  else


    return fib(l->data) + countFib(l->next) ;
}
```

You can assume that no overflow will occur.

Question 5.4 **[10]**

You are given the following two recursive definitions:

$$m(x,y) = \begin{cases} 0, & \text{if } x = 0. \\ m(x-1,y) + y, & \text{otherwise.} \end{cases} \tag{1}$$

$$f(x) = \begin{cases} 1, & \text{if } x = 0. \\ m(\cancel{x}, f(\cancel{x}-1)), & \text{otherwise.} \end{cases} \tag{2}$$

Write two *recursive* functions `m` and `f` by completing the blanks.

```
int m(int x, int y) {

  assert( x>=0 );

  if( x==0 )

    return 0;

  else

    return m(x-1, y) + y;

}
int f( int x ) {

  assert( x>=0 );

  if( x==0 )

    return 1;

  else

    return m(x, f(x-1))

}
```

6 Bonus

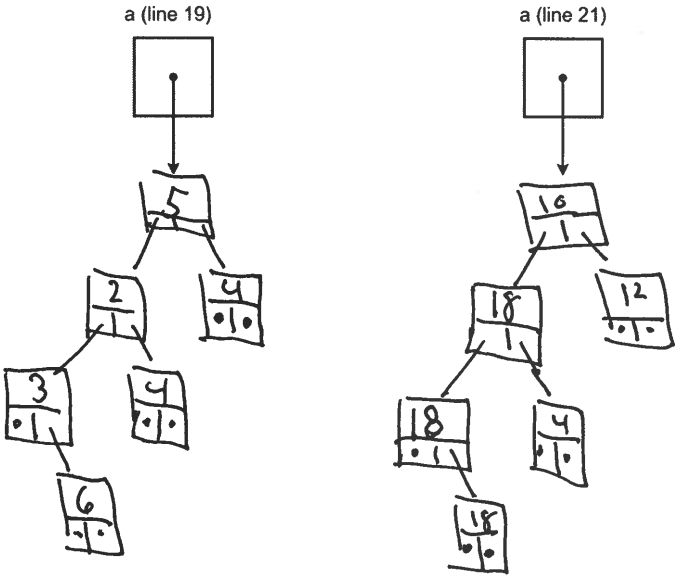
Question 6.1 **[10]**

Consider the following program fragment:

```
typedef struct tree {
   int value;
   struct tree* left;
   struct tree* right;
} tree;


void modify_tree(tree* t) {
   if (t != NULL) {
      if (t->value % 2 == 0) {
         t->value = t->value * 3;
         modify_tree(t->left);
      } else {
         t->value = t->value * 2;
         modify_tree(t->left);
         modify_tree(t->left);
         modify_tree(t->right);
      }
   }
}

tree* new_treenode(int v, tree* l, tree* r) {
   tree* new_t = (tree*)malloc(sizeof(tree));
   new_t->value = v;
   new_t->left = l;
   new_t->right = r;
   return new_t;
}
```

```
int main () {
   tree* a = new_treenode(5,
      new_treenode(2,
         new_treenode(3,
            NULL,
            new_treenode(6,
               NULL,
               NULL)
         ),
         new_treenode(4,
            NULL,
            NULL)
      ),
      new_treenode(4,
         NULL,
         NULL)
   );

  // draw a here
   modify_tree(a);
//draw a here
}
```

Draw, in the same way as drawn in the lectures and question 4.1, the values of the allocated struct (`a`) before and after `modify_tree(a)` has been executed (remember to include `NULL`) on the next page:


a (line 19)
5
2
4
3
4
6
a (line 21)
16
18
12
18
4
18